DSA ki Baithak

CODE BAITHAK

BY NAMAN BHAIYA

How does a computer store numbers ?

* Computer stores data in bits (0 & 1)

* 8 bits = 1 Byte

* Every number → binary form

Positive Number Representation

[get stored directly in binary form]

MSB (Most significant Bit) = 0 → Positive

Ex: int a = 10; → 1010 (4 bits)

2	10	0
2	5	1
2	2	0
2	1	1
	0	

⇒ 1010

MSB → 0 (+ve)
→ 1 (-ve)

32 bits ← (4 bytes)

MSB

0 0 0 0 0 - - - - - - - - - - - - - - - - - - 0 - 0 - 0 - 0 - 0 - - 0 0 0 0 1 0 1 0

+ve

LSB

short c = 5; → 0 - - - 0, 0 0 0 0 0 1 0 1
$\underbrace{}_{8}$ $\underbrace{}_{8}$

(2 bytes)

101

+ve

-5

Negative Number Representation

* Sign-Magnitude Method 1
* 1's Complement Method 2
* 2's Complement Method 3

Method 1 : Sign Magnitude → -ve Number

MSB = sign bit $\begin{bmatrix} 0 \rightarrow +ve \\ 1 \rightarrow -ve \end{bmatrix}$

byte a = 5

$\boxed{0}\,0\,0\,0\,0\,1\,0\,1$

Sign   Magnitude

$\begin{matrix} 0 & 0 \\ 1 & 1 \end{matrix} \rightarrow \left.\begin{matrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{matrix}\right] \rightarrow 4 = 2^2$

$2 \times 2 \times 2 \rightarrow 2^3 = 8$

$n \text{ bit} \rightarrow 2^n$

byte a = -5

$\boxed{1}\,0\,0\,0\,0\,1\,0\,1$

-ve

Sign

byte a = -10

$\boxed{1}\,0\,0\,0\,1\,0\,1\,0$

0

byte a = -10

min

-ve $\boxed{1}$ 1 1 1 1 1 1 1 → 2^8 → 256

+ve $\boxed{0}$ 1 1 1 1 1 1 1

+0, -0

[min, max]

[-127, +127]

min = -127

max = +127

127 + 127 + 1

255

(1), (0)

10 - 1 + 1 = 10

$\boxed{ei - si + 1}$

8 bits

2^8

256

$\boxed{1 \text{ number loss}}$

+ve

(+0) $\boxed{0}$ 0 0 0 0 0 0 0

1 1 1 1 1 1 1

(+127)

[0, 127]

-ve

(-0) $\boxed{1}$ 0 0 0 0 0 0 0

1 1 1 1 1 1 1

-127

[-127, -0]

Method 2 : 1's Complement

101 → 010

Flip all bits $\begin{bmatrix} 0 \rightarrow 1 \\ 1 \rightarrow 0 \end{bmatrix}$

byte a = -5;

+5 → 0 0 0 0 0 1 0 1

↓ 1's comp

1 1 1 1 1 0 1 0 → -5

byte b = -10

+10 → 0 0 0 0 1 0 1 0

↓ 1's comp

1 1 1 1 0 1 0 1 → -10

(+0) 0 0 0 0 0 0 0 0

(+127) 0 1 1 1 1 1 1 1

[0, 127]

+ve

-ve

1 0 0 0 0 0 0 0 → -127 ← 0 1 1 1 1 1 1 1

1 1 1 1 1 1 1 1 → (-0) 0 0 0 0 0 0 0 0

255

256

Method 3 : 2's Complement

[Standard method used in the modern computers]

1. Take 1's Complement
2. Add 1 (binary addition)

$101 \xrightarrow{\text{1's com}} 010$

2's co:

$$\begin{array}{r} 010 \\ +\;\;\;1 \\ \hline 011 \\ \hline \end{array}$$

byte a = -5;

$+5 \longrightarrow$ 0 0 0 0 0 1 0 1

↓ 1's Comp

$$\begin{array}{r} 11111010 \\ +\qquad\quad 1 \\ \hline \boxed{11111011} \end{array}$$

-5

$$\begin{array}{r} 00000100 \\ +\qquad\quad 1 \\ \hline \boxed{00000101} \end{array}$$

byte a = -10;

+10 → 00001010

↓ 1's Comp

11110101

$+\quad 1$

$\boxed{11110110}$

↓ 1's comp

00001001

$+\quad 1$

$\boxed{00001010}$

-10

$256 \rightarrow 2^8$

_ _ _ _ _ _ _ _

+ve   -ve

(+0) $\boxed{0}$0000000

(+127) 01111111

[0, 127]

128 → +ve

$\boxed{1}$0000000 → 10000000 (-128)

11111111 → 00000001

(-1) -ve

[-128, -1]

-1 + 128 + 1 = 128

n bits → 2^n

+ve: $\dfrac{2^n}{2} \rightarrow 2^{n-1}$

-ve: $\dfrac{2^n}{2} \rightarrow 2^{n-1}$

8 bits $\rightarrow 2^8 = 256$

$\dfrac{2^{\cancel{8}7}}{\cancel{2}}$ $\qquad$ $\dfrac{2^8}{2}$

$2^7 - 1$

$[0, 127]$ $\qquad$ $2^7 \rightarrow$ +ve $\qquad$ $2^7 \rightarrow$ -ve $\qquad$ $[-128, -1]$

128 $\qquad$ 128

2^{n-1} $\qquad$ $[0, (2^{n-1} - 1)]$ $\qquad$ $[-2^{n-1}, -1]$

2^{n-1}

+ve $\rightarrow [0, 127]$

-ve $\rightarrow [-128, -1]$

$\Downarrow$

$[-128, 127]$

Range of Numbers / Range Formula

For 'n' bits :

$[-2^{n-1}, (2^{n-1}-1)]$

$$\text{Range} = -2^{n-1} \text{ to } (2^{n-1}-1)$$

ie, $+ve \rightarrow 0$ to $(2^{n-1}-1)$ ← $(2^{n-1}$ numbers$)$

$-ve \rightarrow -2^{n-1}$ to -1

[Half Range +ve &
Half Range -ve]

Data Type Ranges in Java $\left[-2^{n-1} \text{ to } (2^{n-1}-1) \right]$

$n \to 8$

* byte (8 bits) : -2^{7} to $(2^{7}-1)$ $\to$ $\boxed{-128 \text{ to } 127}$

$\to$ 128 ✗

✓

$n \to 16$

* short (16 bits) : -2^{15} to $(2^{15}-1)$

* int (32 bits) : -2^{31} to $(2^{31}-1)$

* long (64 bits) : -2^{63} to $(2^{63}-1)$

⋮